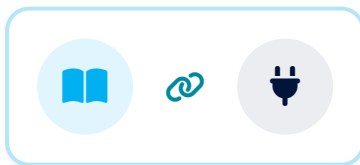


 DEVELOPER GUIDE

How to Actually Read API Documentation

Endpoints, auth, rate limits, and request/response structure:
decoded.



Swipe to master API docs →



AI Coach John



Follow

 Repost

Why This Skill Matters

Every AI project eventually needs this. Whether you are calling OpenAI, Anthropic, a vector database, or any third-party service, you are reading API docs. Most beginners skim docs, guess at the code, and get stuck on cryptic errors that the documentation actually explained clearly.

🔧 WHAT WE WILL DECODE SECTION BY SECTION

01

Endpoints & Methods

Actions to trigger

02

Authentication

Proving who you are

03

Request Structure

Headers vs data body

04

Response Structure

Status & JSON dict

05

Status Codes & Errors

Instant error triage

06

Rate Limits & Backoff

Reading metric headers

07

Pagination & Versioning

Handling large lists & pinning API versions for production



AI Coach John



PROITBRIDGE
Strive for Better Future

Follow

 Repost

Anatomy of API Documentation

Every good API doc page has the same skeleton. Once you know these 7 sections always show up in some form, unfamiliar API docs stop feeling overwhelming. You are just filling in the same template with different values.

Endpoint

→ What you are calling and how

Description

→ What exact action it performs

Authentication

→ How you prove who you are

Request body

→ What data payload you send

Response

→ What exact data you get back

Status codes

→ What different outcomes mean

Rate limits

→ How many times you can call it



Look for these exact 7 blocks whenever opening a new API reference page.

**AI Coach John****Follow** **Repost**

Endpoints & HTTP Methods

An **endpoint** is a specific URL path that performs one action. The **HTTP method** tells the server what kind of action you want to execute:

GET

Retrieve data

GET /v1/users/123

POST

Create something new

POST /v1/messages

PUT / PATCH

Update existing resource

PUT /v1/users/123

DELETE

Remove a resource

DELETE /v1/users/123

! CRITICAL DISTINCTION

The same base path supports different actions depending on the method.

Always check both path AND method in the docs: `GET /v1/messages` and `POST /v1/messages` are completely different operations!



AI Coach John



Follow

 Repost

Base URL, Path & Query Parameters

Reading the full URL correctly: Every API address breaks down into structural components that serve distinct purposes:

```
https://api.example.com/v1/users/123?include=orders&limit=10
```

base URL path path query parameters
parameter

Base URL

The root address for all requests to this API (often shown once at the top of the docs).

Path parameter

Part of the URL itself, usually an ID (`/users/123` where `123` identifies which user).

Query params

Key-value pairs after `?`, used for filtering, pagination, or optional settings.

```
import requests

response = requests.get(
    "https://api.example.com/v1/users/123",
    params={"include": "orders", "limit": 10}
)
# requests automatically builds query string
```



Docs show path parameters like `/users/{user_id}`. That is a placeholder you replace with a real ID value.



AI Coach John



Follow

 Repost

Authentication: The 3 Types

Proving who you are to the API: Almost all modern APIs use one of three core credential mechanisms:

API KEY

A secret string sent with every request, often in headers or parameters.

Header → X-API-Key: sk-abc123 ...

BEARER TOKEN

Secret string sent in the **Authorization** header prefixed with "Bearer".

Header → Authorization: Bearer sk-ant-abc ...

OAUTH 2.0

Multi-step flow exchanging credentials for a temporary access token.

Header → Authorization: Bearer ya29.a0AfH ...



Exact header name varies by provider (`X-API-Key`, `api-key`, `Authorization`). Always check the auth section first!



AI Coach John



Follow

 Repost

Making an **Authenticated Request**

Putting auth into actual code: Send a real request reading secrets straight from environment variables (`dotenv`). Never hardcode API keys!

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()
api_key = os.getenv("API_KEY")

response = requests.post(
    "https://api.example.com/v1/messages",
    headers={
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json"
    },
    json={"message": "Hello"}
)

print(response.status_code)
print(response.json())
```

A WHY CONTENT-TYPE MATTERS

Content-Type: application/json tells the server your payload format. Forgetting this header is the #1 reason a technically correct request fails!



Request: Headers vs Body

What actually goes where: Every HTTP POST/PUT request separates configuration metadata from data payload.

H HEADERS

Metadata about request: auth token, content type format, API version string.

📦 BODY (JSON)

Actual data payload sent with POST/PUT (`model`, `messages`, `max\_tokens`).

```
headers = {  
  "Authorization": "Bearer sk-abc123 ...",  
  "Content-Type": "application/json",  
  "Anthropic-Version": "2023-06-01" # custom header  
}  
  
body = {  
  "model": "claude-sonnet-4-6",  
  "max_tokens": 1000,  
  "messages": [{"role": "user", "content": "Hello"}]  
}  
  
response = requests.post(url, headers=headers, json=body)
```



Match exact field names from docs in `json=body`. APIs strictly reject mismatched nesting!



AI Coach John



Follow

 Repost

Response Structure

What comes back, and how to read it: Every response returned by the server has three functional components:

Status Code

Number ('200')

≡ Headers

Rate limits, ID

📄 Body

JSON Data Dict

```
response = requests.post(url, headers=headers, json=body)

print(response.status_code) # e.g., 200
print(response.headers)    # e.g., {'content-type': '...'}
print(response.json())     # parsed dict
```

```
{
  "id": "msg_123",
  "content": [{"type": "text", "text": "Hello!"}],
  "usage": {"input_tokens": 8, "output_tokens": 12}
}
```



Always verify `status_code == 200` before calling `.json()`. HTML error pages cause JSON decode errors!



AI Coach John



Follow

 Repost

HTTP Status Codes Cheat Sheet

What the number actually means: The first digit reveals the category (`2xx` success, `4xx` client mistake, `5xx` server outage):

200**Success:** Parse the response body normally.**201****Created:** New resource successfully created.**400****Bad Request:** Malformed JSON body or parameters.**401****Unauthorized:** Auth missing or invalid API key.**403****Forbidden:** Key valid, but permission denied.**404****Not Found:** Endpoint URL does not exist.**429****Rate Limited:** Too many calls; back off and retry.**500+****Server Error:** Provider issue; retry with backoff.

Check the first digit instantly: you can triage errors without reading a single line of body text!

**AI Coach John****Follow** **Repost**

Rate Limits: **Reading Headers**

How much you are allowed to call the API: Most APIs return real-time rate limit tracking directly in HTTP response headers:

LIMIT

50

Max / minute

REMAINING

12

Calls left

RESET

17215584

Unix timestamp

Typical Response Headers returned by API:

X-RateLimit-Limit: 50

X-RateLimit-Remaining: 12

X-RateLimit-Reset: 1721558400

```
response = requests.get(url, headers=headers)
rem = response.headers.get("X-RateLimit-Remaining")
rst = response.headers.get("X-RateLimit-Reset")
print(f"{rem} requests remaining, resets at {rst}")
```



Reading headers proactively lets your script slow down before you actually get rate limited!



AI Coach John



Follow

 Repost

Handling Limits: **Retry + Backoff**

What to do when you hit a 429: Automatically retry after waiting with exponentially increasing delays (the industry standard pattern):

Attempt 1

Wait 1s

**Attempt 2**

Wait 2s

**Attempt 3**

Wait 4s

**Attempt 4**

Wait 8s

```
import time

def call_with_backoff(url, headers, body, max_retries=5):
    for attempt in range(max_retries):
        response = requests.post(url, headers=headers,
                                json=body)
        if response.status_code == 429:
            wait_time = 2 ** attempt # 1s, 2s, 4s, 8s, 16s...
            print(f"Rate limited. Waiting {wait_time}s...")
            time.sleep(wait_time)
            continue
        return response
    raise Exception("Max retries exceeded.")
```



Exponential backoff prevents hammering overloaded servers. Official SDKs often build this in automatically!

**AI Coach John****Follow** **Repost**

Pagination: Handling Lists

When a response has too much data for one call: APIs split list results into pages across separate network requests:

OFFSET / LIMIT

```
?limit=20&offset=40
```

Skip first 40 items, return next 20. Simple, but can skip items if DB changes.

CURSOR-BASED

```
?cursor=eyJpZCI6NDJ9
```

Pass ``next_cursor`` from previous call. More reliable for real-time live data.

```
def get_all_items(url, headers):
    items = []
    cursor = None
    while True:
        params = {"cursor": cursor} if cursor else {}
        resp = requests.get(url, headers=headers,
params=params).json()
        items.extend(resp["data"])
        cursor = resp.get("next_cursor")
        if not cursor:
            break
    return items
```



Always check pagination docs when querying lists (`users`, `messages`).
Default pages cap at 20 or 50!



AI Coach John



Follow

 Repost



API Versioning: Pinning Stability

Why the version matters: APIs evolve constantly. Versioning lets providers release breaking changes without disrupting existing apps. Look for versioning in 3 places:

URL Path

`https://api.example.com/v1/messages`

Custom Header

`Anthropic-Version: 2023-06-01`

Accept Header

`Accept: application/vnd.api+json;version=2`

⚠ NEVER RELY ON DEFAULTS

Always explicitly pin the API version in your request headers or URL path rather than relying on "whatever the default is". Providers change defaults over time, which can silently break unpinned production integrations!



Reading a Real Example: **End to End**

Putting every piece together: Run this exact mental checklist when reading any API reference page:

Endpoint: POST /v1/messages

Auth: Bearer token

Headers: JSON + Version

Limits: 50 calls/min

```
response = requests.post(
    "https://api.example.com/v1/messages",
    headers={
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
        "Anthropic-Version": "2023-06-01"
    },
    json={
        "model": "claude-sonnet-4-6",
        "max_tokens": 1000,
        "messages": [{"role": "user", "content": "Hello"}]
    }
)

if response.status_code == 200:
    print(response.json()["content"])
else:
    print(f"Error {response.status_code}: {response.json()}")
```

☑️ Checklist complete: Method + Auth + Headers + Body + Status Handling!



AI Coach John



Follow

🔄 Repost

Common Pitfalls to Avoid

Mistakes that cause confusing errors: Avoid these 6 frequent developer traps:

-  **Missing Content-Type:** Forgetting ``application/json`` causes servers to misinterpret your body.
-  **Blind JSON Parsing:** Calling ``json()`` before checking ``status_code`` hides real errors behind parse crashes.
-  **Path vs Query Mixup:** ``{user_id}`` in path is not the same as ``?user_id=...`` parameter string.
-  **Hardcoded Limits:** Guessing rate limits instead of reading ``X-RateLimit`` response headers.
-  **Unpinned Versions:** Relying on default API versions that can change and break integration.
-  **Ignoring Pagination:** Assuming one request got "everything" when list endpoints cap at 20 to 50 items.

